

小结

C++语言可以用于解决各种类型的问题，既有几个小时就可以解决的小问题，也有一个大团队工作数年才能解决的超大规模问题。C++的某些特性特别适合于处理超大规模问题，这些特性包括：异常处理、命名空间以及多重继承或虚继承。

异常处理使得我们可以将程序的错误检测部分与错误处理部分分隔开来。当程序抛出一个异常时，当前正在执行的函数暂时中止，开始查找最邻近的与异常匹配的 `catch` 语句。作为异常处理的一部分，如果查找 `catch` 语句的过程中退出了某些函数，则函数中定义的局部变量也随之销毁。

命名空间是一种管理大规模复杂应用程序的机制，这些应用可能是由多个独立的供应商分别编写的代码组合而成的。一个命名空间是一个作用域，我们可以在其中定义对象、类型、函数、模板以及其他命名空间。标准库定义在名为 `std` 的命名空间中。

从概念上来说，多重继承非常简单：一个派生类可以从多个直接基类继承而来。在派生类对象中既包含派生类部分，也包含与每个基类对应的基类部分。虽然看起来很简单，但实际上多重继承的细节非常复杂。特别是对多个基类的继承可能会引入新的名字冲突，并造成来自于基类部分的名字的二义性问题。

如果一个类是从多个基类直接继承而来的，那么有可能这些基类本身又共享了另一个基类。在这种情况下，中间类可以选择使用虚继承，从而声明愿意与层次中虚继承同一基类的其他类共享虚基类。用这种方法，后代派生类中将只有一个共享虚基类的副本。

术语表

捕获所有异常 (catch-all) 异常声明形如 (...) 的 `catch` 子句。一条捕获所有异常的子句可以捕获任意类型的异常。常用于捕获局部检测的异常，该异常将重新抛出到程序的其他部分并最终解决问题。

catch 子句 (catch clause) 程序中负责处理异常的部分。`catch` 子句包含关键字 `catch`，后面是异常声明以及一个语句块。`catch` 子句的代码负责处理异常声明中定义的异常。

构造函数顺序 (constructor order) 在非虚继承中，基类的构造顺序与其在派生列表中出现的顺序一致。在虚继承中，首先构造虚基类。虚基类的构造顺序与其在派生类的派生列表中出现的顺序一致。只有最低层的派生类才能初始化虚基类。虚基类的初始值如果出现在中间基类中，则这些初始值将被忽略。

异常声明 (exception declaration) `catch` 子句中指定其能够处理的异常类型的部分。异常声明的行为与形参列表类似，其中的唯一一个形参通过异常对象进行初始化。如果异常说明符是非引用类型，则异常对象将被拷贝给 `catch`。

异常处理 (exception handling) 管理运行时异常的语言级支持。代码中一个独立开发的部分可以检测并引发异常，由程序的另一个独立开发的部分处理该异常。也就是说，程序的错误检测部分负责抛出异常，而错误处理部分在 `try` 语句块的 `catch` 子句中处理异常。

异常对象 (exception object) 用于在异常的 `throw` 和 `catch` 之间进行通信的对象。在抛出点创建该对象，该对象是被抛出的表达式的副本。在该异常的最后一段处理代码完成之前异常对象都一直存在。异常对象的类型是被抛出的表达式的静态类型。

文件中的静态声明 (file static) 使用关键字 `static` 声明的仅对当前文件有效的名字。在 C 语言和之前的 C++ 版本中, 文件中的静态声明用于声明只能在当前文件中使用的名字。该特性在当前的 C++ 版本中已经被未命名的命名空间替换了。

函数 try 语句块 (function try block) 用于捕获构造函数初始化过程发生的异常。关键字 `try` 出现在表示构造函数初始值列表开始的冒号之前 (或者当初始值列表为空时出现在函数体的左侧花括号之前), 并以函数体右侧花括号之后的一个或几个 `catch` 子句作为结束。

全局命名空间 (global namespace) 是每个程序的隐式命名空间, 用于存放全局定义。

处理代码 (handler) 是“`catch` 子句”的同义词。

内联的命名空间 (inline namespace) 内联命名空间中的名字可以看成是外层命名空间的成员。

多重继承 (multiple inheritance) 有多个直接基类的类。派生类继承所有基类的成员。可以为每个基类分别设定访问说明符。

命名空间 (namespace) 将库或者其他程序集定义的名字放在同一个作用域中的机制。和 C++ 的其他作用域不同, 命名空间作用域可以定义成几个部分。我们可以打开并关闭命名空间, 然后在程序的另一个地方重新打开并关闭该命名空间。

命名空间的别名 (namespace alias) 为某个给定的命名空间定义同义词的机制:

```
namespace N1 = N;
```

将 `N1` 定义成命名空间 `N` 的另一个名字。命名空间可以含有多个别名, 命名空间的原名和别名是等价的。

命名空间污染 (namespace pollution) 当所有类和函数的名字都放置于全局命名空间时将造成命名空间污染。如果来自于多个独立供应商的代码都含有全局名字, 则使用这些代码的大型程序很可能会面临命

名空间污染的问题。

noexcept 运算符 (noexcept operator) 该运算符返回一个 `bool` 值, 用于表示给定的表达式是否会抛出异常。该表达式不会被求值, 运算的结果是一个常量表达式。当提供的表达式不含 `throw` 并且只调用了做出不抛出说明的函数时, 结果为 `true`; 否则结果为 `false`。

noexcept 说明 (noexcept specification) 表示函数是否会抛出异常的关键字。当 `noexcept` 跟在函数的形参列表之后时, 它可以连接一个括号括起来的常量表达式, 前提是该表达式可以转换成 `bool` 值。如果忽略了该表达式, 或者表达式的值为 `true`, 则函数不会抛出异常。如果表达式的值是 `false` 或者函数没有异常声明, 则其可能抛出异常。

不抛出说明 (nothrow specification) 该异常说明用于承诺某个函数不会抛出异常。如果一个做了不抛出说明的函数实际抛出了异常, 将调用 `terminate`。不抛出说明符是不含实参或者含有一个值为 `true` 的实参的 `noexcept`。

引发 (raise) 常常作为抛出的同义词。C++ 程序员认为抛出异常和引发异常基本上是等价的。

重新抛出 (rethrow) 不指定表达式的 `throw`。重新抛出只有在 `catch` 子句内部或者被 `catch` 直接或间接调用了的函数内时才有效。它的效果是将其接受的异常重新抛出。

栈展开 (stack unwinding) 在搜寻 `catch` 时依次退出函数的过程。异常发生前构造的局部对象将在进入相应的 `catch` 前被销毁。

terminate 是一个标准库函数, 当异常未被捕获或者在处理异常的过程中发生了另一个异常时, `terminate` 负责结束程序的执行。

throw e 该表达式将中断当前的执行路径, `throw` 语句将控制权传递给最近的能够处

理该异常的 catch 子句。表达式 e 将被拷贝给异常对象。

try 语句块 (try block) 含有关键字 try 以及一个或多个 catch 子句的语句块。如果 try 语句块中的代码引发了一个异常，并且某个 catch 可以匹配该异常，则异常将被这个 catch 处理。否则，异常被传递到 try 语句块之外并继续沿着调用链寻找与之匹配的 catch。

未命名的命名空间 (unnamed namespace) 定义时未指定名字的命名空间。对于定义在未命名的命名空间中的名字，我们可以不用作用域运算符就直接访问它们。每个文件有一个独有的未命名的命名空间，其中的名字在文件外不可见。

using 声明 (using declaration) 是一种将命名空间中的某个名字注入当前作用域的机制：

```
using std::cout;
```

上述语句使得命名空间 std 中的名字 cout 在当前作用域可见。之后，我们就可以直接使用 cout 而无须前缀 std::了。

using 指示 (using directive) 是具有如下形式的声明：

```
using NS;
```

上述语句使得命名空间 NS 的所有名字在 using 指示所在的作用域以及 NS 所在的作用域都变得可见。

虚基类 (virtual base class) 在派生列表中使用了关键字 virtual 的基类。在派生类对象中，虚基类部分只有一份，即使该虚基类在继承体系中出现了多次也是如此。对于非虚继承而言，构造函数只能初始化它的直接基类。但是对于虚继承来说，虚基类将被最低层的派生类初始化，因此最低层的派生类应该含有它的所有虚基类的初始值。

虚继承 (virtual inheritance) 是多重继承的一种形式，基类被继承了多次，但是派生类共享该基类的唯一一份副本。

作用域运算符 (:: operator) 用于访问命名空间或类中的名字。

第 19 章

特殊工具与技术

内容

19.1 控制内存分配	726
19.2 运行时类型识别	730
19.3 枚举类型	736
19.4 类成员指针	739
19.5 嵌套类	746
19.6 union: 一种节省空间的类	749
19.7 局部类	754
19.8 固有的不可移植的特性	755
小结	762
术语表	762

本书的前三部分讨论了 C++ 语言的基本要素，这些要素绝大多数程序员都会用到。此外，C++ 还定义了一些非常特殊的性质，对于很多程序员来说，他们一般很少会用到本章介绍的内容。